Tier 1: What runs on the box

Pick your runtime by hardware, not preference.

Runtime	Best fit
ONNX Runtime	Mixed fleets. Broadest execution providers: CUDA, TensorRT, OpenVINO, NNAPI, CoreML
LiteRT	Mobile and embedded. Successor to TensorFlow Lite, with LiteRT-LM for on-device language models
OpenVINO	Intel CPUs, GPUs, NPUs. NNCF INT8 routinely gives 2x to 3x speedup
TensorRT	NVIDIA Jetson. Fastest on NVIDIA silicon; least portable
llama.cpp	On-device LLMs on almost anything

If your fleet is genuinely heterogeneous, which most are after two refresh cycles, ONNX Runtime is the pragmatic default. You pay in footprint and buy the ability to ship one artefact to three hardware generations.

Quantization is not optional here. INT8 typically gives a two to threefold speedup at roughly a quarter the model size, with accuracy loss often within noise for detection work. Measure on your own data but expect to ship INT8.

Two things the device tier must do beyond inference, and both are forgotten in prototypes. Buffer locally, because when the uplink drops, results need somewhere to go, with a defined policy for what happens when the buffer fills. And report health unprompted, since pull-based monitoring does not work across NAT, cellular links, or devices that sleep.

Tier 2: Orchestration without the cloud

K3s is CNCF-conformant Kubernetes in one binary, running in 512MB to 1GB for a server node and around 256MB for an agent, with ARM and x86 parity. For most sites with a reasonable box, this is the answer.

KubeEdge splits into CloudCore beside your API server and EdgeCore on edge nodes, caching state in SQLite and running workloads autonomously when the link is down. It has more moving parts than K3s, which is worth it when offline autonomy is a hard requirement.

Akri, a CNCF Sandbox project, exposes leaf devices such as IP cameras as Kubernetes resources, handling their appearance and disappearance. If your workload is fundamentally cameras, it removes a lot of glue code.

These compose. A Fleet-managed K3s cluster at a regional aggregation point can host CloudCore for devices beneath it.

On GitOps: Push-based deployment fails at the edge for the obvious reason. The pull model inverts it. Each site runs an agent that pulls desired state from Git when connectivity returns and reconciles itself. Rancher Fleet, Argo CD, and Flux all do this, and your source of truth stays a repository of plain manifests.

```
yaml
defaultNamespace: edge-inference
helm:
  chart: ./charts/vision-inference
  values:
    modelVersion: "2026.09.3"
    inferenceRuntime: onnxruntime
targetCustomizations:
  - name: jetson-sites
    clusterSelector:
      matchLabels: { hardware: jetson-orin }
    helm:
      values: { inferenceRuntime: tensorrt }
  - name: intel-sites
    clusterSelector:
      matchLabels: { hardware: intel-nuc }
    helm:
      values: { inferenceRuntime: openvino }
```

One definition -- hardware overrides by label. That pattern is most of what fleet management is.

Tier 3: Models are not container images

This is the part teams get wrong most often. The instinct is to bake the model into the image. It works until about fifty devices, then fails for three reasons. Updates require a full rebuild and redeploy. A 400MB image pushed to four thousand devices over cellular is a bandwidth bill someone will query. And you lose the ability to roll back the model independently of application code, which is exactly the rollback you want at 2 am.

Separate them.

```
registry.internal/edge-inference:1.8.2
# application, changes rarely

models.internal/defect-detect/2026.09.3.onnx
# model, changes often
```

The device caches the current and previous. Rollback becomes a config change completing in seconds with no network.

Staged rollout is mandatory, not a refinement:

1. canary 5 devices, 48 hours, chosen for hardware diversity
2. pilot 1 site, 1 week, human validating output
3. ring 1 10% of fleet
4. ring 2 50
5. full remainder

Design an automatic halt on an error budget at every stage. Those 48 hours matter more than they look, because edge failures are often environmental and need a full day-night cycle to surface. A model that behaves at 2 pm can fall apart under fluorescent light at 11 pm.